PYTHON PROGRAMMING CASE STUDIES

Problem • Algorithm • Python Code • Sample Output • Explanation

10 Selected Activity-Based Learning Case Studies

Student Information

Student Name: Polagani Ramya Sri

Roll Number: 26000192

Section: CSE-3

Case Studies Included

1. Progressive Electricity Billing
2. Hospital Billing and Patient Classification
3. Employee Payroll and Deduction Analysis
4. Retail Billing with Discount Rules
5. Examination Result and Grace-Mark Validation
6. Attendance Recovery Computation
7. ATM Transaction Sequence Simulation
8. Warehouse Inventory Reconciliation
9. Advanced Numeric Pattern Generator
10. Student Marks Matrix Analysis

Python Activity-Based Learning

Includes source code, test cases, sample outputs and conclusions for all 10 case studies.

Contents

1. Progressive Electricity Billing

3. Hospital Billing and Patient Classification

4. Employee Payroll and Deduction Analysis

7. Retail Billing with Discount Rules

11. Examination Result and Grace-Mark Validation

12. Attendance Recovery Computation

13. ATM Transaction Sequence Simulation

14. Warehouse Inventory Reconciliation

27. Advanced Numeric Pattern Generator

42. Student Marks Matrix Analysis

How to use this document

Run each Python program in Python IDLE, VS Code, or another Python 3 environment. The displayed output is a sample run; your output will depend on the values entered. Flowcharts are written as clear text flows so they can be redrawn as standard flowchart symbols if required.

Case Study 1: Progressive Electricity Billing

1. Problem Statement

Create a program that calculates a household electricity bill using progressive consumption slabs, a fixed charge, and a rebate for low bills.

2. Objectives

• Calculate the energy charge using different unit slabs.

• Add the fixed charge and apply the stated rebate rule.

• Display the units used and the final bill.

3. Assumptions

• The first 100 units cost ■2 per unit.

• Units 101–200 cost ■3 per unit.

• Units above 200 cost ■5 per unit.

• A fixed charge of ■50 is added.

• If the bill before rebate is below ■300, a rebate of ■20 is applied.

• Units cannot be negative.

4. Algorithm

• Start.

• Read electricity units.

• If units are negative, display an invalid-input message.

• Calculate the charge using the progressive slabs.

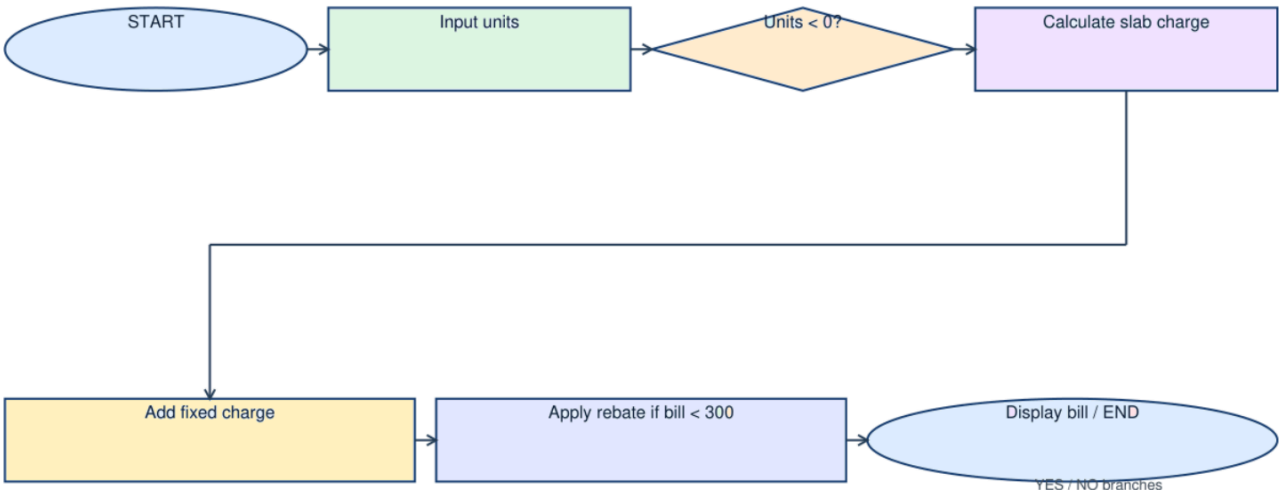
• Add the fixed charge.

• If the bill is below ■300, subtract ■20.

• Display the final bill.

• Stop.

5. Flowchart



6. Pseudocode

```
BEGIN
  INPUT units
  IF units < 0 THEN
```

```
      PRINT "Invalid units"
  ELSE
      IF units <= 100 THEN
          energy_charge = units * 2
      ELSE IF units <= 200 THEN
          energy_charge = 100 * 2 + (units - 100) * 3
      ELSE
          energy_charge = 100 * 2 + 100 * 3 + (units - 200) * 5
      END IF
      bill = energy_charge + 50
      IF bill < 300 THEN bill = bill - 20
      PRINT bill
  END IF
END
```

7. Python Source Code

```
units = int(input("Enter electricity units: "))

if units < 0:
    print("Invalid units")
else:
    if units <= 100:
        energy_charge = units * 2
    elif units <= 200:
        energy_charge = 100 * 2 + (units - 100) * 3
    else:
        energy_charge = 100 * 2 + 100 * 3 + (units - 200) * 5

    bill = energy_charge + 50

    if bill < 300:
        bill = bill - 20

    print("Units consumed:", units)
    print("Final electricity bill: Rs.", round(bill, 2))
```

8. Test Cases

Units	Expected result
50	■130 (■100 energy + ■50 fixed – ■20 rebate)
150	■400
250	■850
-5	Invalid units

9. Sample Output

```
Enter electricity units: 150
Units consumed: 150
Final electricity bill: Rs. 400
```

10. Conclusion

The program uses conditional statements to calculate a bill with progressive slabs, a fixed charge, and a rebate.

Case Study 2: Hospital Billing and Patient Classification

1. Problem Statement

Calculate a patient's hospital bill from consultation, diagnostic, and treatment charges, then apply a concession based on the total charge.

2. Objectives

• Add all hospital charges.

• Apply a concession when the bill reaches the stated threshold.

• Classify the final bill into a simple payment category.

3. Assumptions

• A 10% concession is given when the subtotal is ■5,000 or more.

• The final bill below ■2,000 is classified as Low; ■2,000–■4,999 as Medium; ■5,000 or more as High.

• All charges must be zero or positive.

4. Algorithm

• Start and read the three charges.

• Validate that none of the charges is negative.

• Add the charges to find the subtotal.

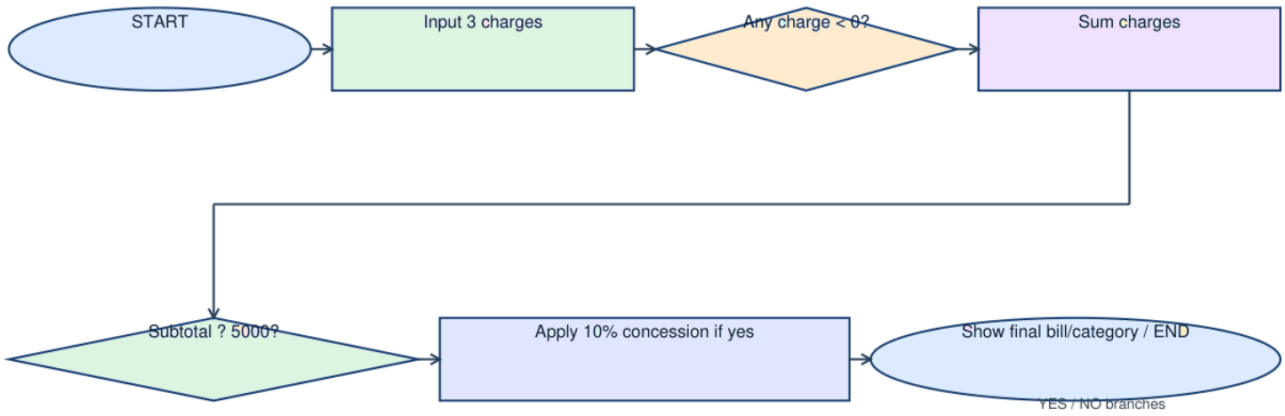
• If subtotal is at least ■5,000, calculate a 10% concession.

• Subtract the concession and classify the final bill.

• Display subtotal, concession, final bill, and category.

• Stop.

5. Flowchart



6. Pseudocode

```
BEGIN
  INPUT consultation, diagnostics, treatment
  IF any charge < 0 THEN PRINT "Invalid charges"
  ELSE
      subtotal = consultation + diagnostics + treatment
      IF subtotal >= 5000 THEN concession = subtotal * 0.10
      ELSE concession = 0
      END IF
      final_bill = subtotal - concession
      IF final_bill < 2000 THEN category = "Low"
      ELSE IF final_bill < 5000 THEN category = "Medium"
      ELSE category = "High"
      PRINT subtotal, concession, final_bill, category
  END IF
```

```
END
```

7. Python Source Code

```
consultation = float(input("Consultation charge: "))
diagnostics = float(input("Diagnostic charges: "))
treatment = float(input("Treatment charges: "))

if consultation < 0 or diagnostics < 0 or treatment < 0:
    print("Invalid charges")
else:
    subtotal = consultation + diagnostics + treatment

    if subtotal >= 5000:
        concession = subtotal * 0.10
    else:
        concession = 0

    final_bill = subtotal - concession

    if final_bill < 2000:
        category = "Low"
    elif final_bill < 5000:
        category = "Medium"
    else:
        category = "High"

    print("Subtotal: Rs.", round(subtotal, 2))
    print("Concession: Rs.", round(concession, 2))
    print("Final bill: Rs.", round(final_bill, 2))
    print("Payment category:", category)
```

8. Test Cases

Consultation, diagnostics, treatment	Expected result
500, 300, 700	Subtotal ■1,500; final ■1,500; Low
1000, 1500, 3000	Subtotal ■5,500; concession ■550; final ■4,950; Medium
-100, 200, 300	Invalid charges

9. Sample Output

```
Consultation charge: 1000
Diagnostic charges: 1500
Treatment charges: 3000
Subtotal: Rs. 5500.0
Concession: Rs. 550.0
Final bill: Rs. 4950.0
Payment category: Medium
```

10. Conclusion

The program combines arithmetic and conditional statements to calculate hospital charges, apply concessions, and classify the payment amount.

Case Study 3: Employee Payroll and Deduction Analysis

1. Problem Statement

Calculate an employee's gross salary from basic pay and allowances, apply a simple deduction rule, and print a salary statement.

2. Objectives

• Calculate allowances and gross salary.

• Apply a deduction based on gross salary.

• Display gross salary, deduction, and take-home salary.

3. Assumptions

• House rent allowance (HRA) is 20% of basic pay.

• Other allowance is 10% of basic pay.

• If gross salary is ■30,000 or more, the deduction is 8% of gross salary; otherwise it is 5%.

• Basic pay must be positive.

4. Algorithm

• Read basic pay.

• If basic pay is not positive, show an error.

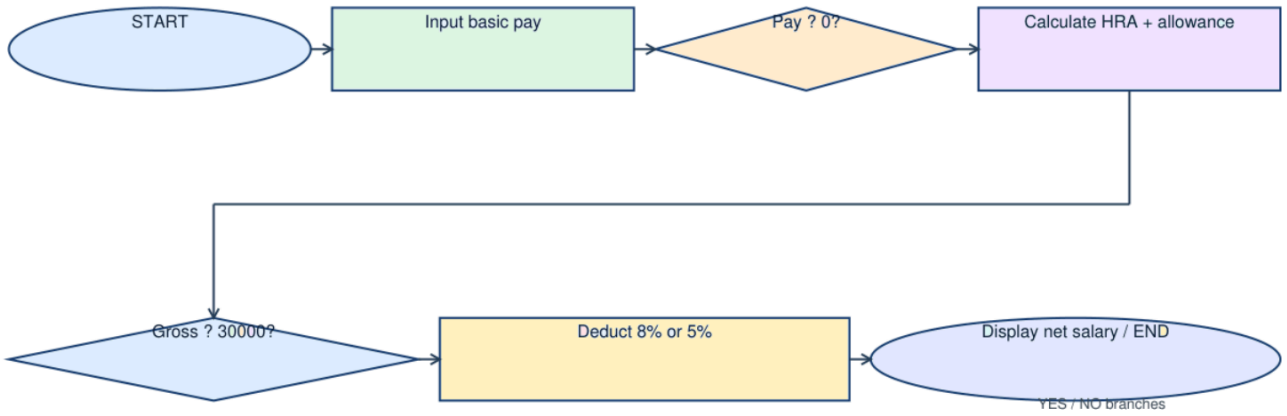
• Calculate HRA and other allowance.

• Calculate gross salary.

• Calculate deduction using the threshold rule.

• Calculate net salary and print the statement.

5. Flowchart



6. Pseudocode

```
BEGIN
  INPUT basic
  IF basic <= 0 THEN PRINT "Invalid basic pay"
  ELSE
      hra = basic * 0.20
      other = basic * 0.10
      gross = basic + hra + other
      IF gross >= 30000 THEN rate = 0.08 ELSE rate = 0.05
      deduction = gross * rate
      net = gross - deduction
      PRINT basic, hra, other, gross, deduction, net
  END IF
END
```

7. Python Source Code

```
basic = float(input("Enter basic pay: "))

if basic <= 0:
    print("Invalid basic pay")
else:
    hra = basic * 0.20
    other_allowance = basic * 0.10
    gross = basic + hra + other_allowance

    if gross >= 30000:
        deduction_rate = 0.08
    else:
        deduction_rate = 0.05

    deduction = gross * deduction_rate
    net_salary = gross - deduction

    print("\n----- SALARY STATEMENT -----")
    print("Basic pay: Rs.", round(basic, 2))
    print("HRA: Rs.", round(hra, 2))
    print("Other allowance: Rs.", round(other_allowance, 2))
    print("Gross salary: Rs.", round(gross, 2))
    print("Deduction: Rs.", round(deduction, 2))
    print("Take-home salary: Rs.", round(net_salary, 2))
```

8. Test Cases

Basic pay	Expected result
20000	Gross ■26,000; deduction ■1,300; net ■24,700
25000	Gross ■32,500; deduction ■2,600; net ■29,900
0	Invalid basic pay

9. Sample Output

```
Enter basic pay: 20000

----- SALARY STATEMENT -----
Basic pay: Rs. 20000.0
HRA: Rs. 4000.0
Other allowance: Rs. 2000.0
Gross salary: Rs. 26000.0
Deduction: Rs. 1300.0
Take-home salary: Rs. 24700.0
```

10. Conclusion

The program demonstrates salary calculations, percentage-based allowances, conditional deductions, and formatted output.

Case Study 4: Retail Billing with Discount Rules

1. Problem Statement

Generate an itemized retail bill for several products, apply a threshold discount, and add tax to the discounted amount.

2. Objectives

• Calculate each product's total cost.

• Apply a discount based on the subtotal.

• Calculate tax and the final amount.

3. Assumptions

• The user enters three product prices and quantities.

• All prices must be positive and quantities must be greater than zero.

• A 10% discount applies when subtotal is at least ■2,000; otherwise no discount applies.

• Tax is 5% of the amount after discount.

4. Algorithm

• Read the price and quantity for three products.

• Validate the values.

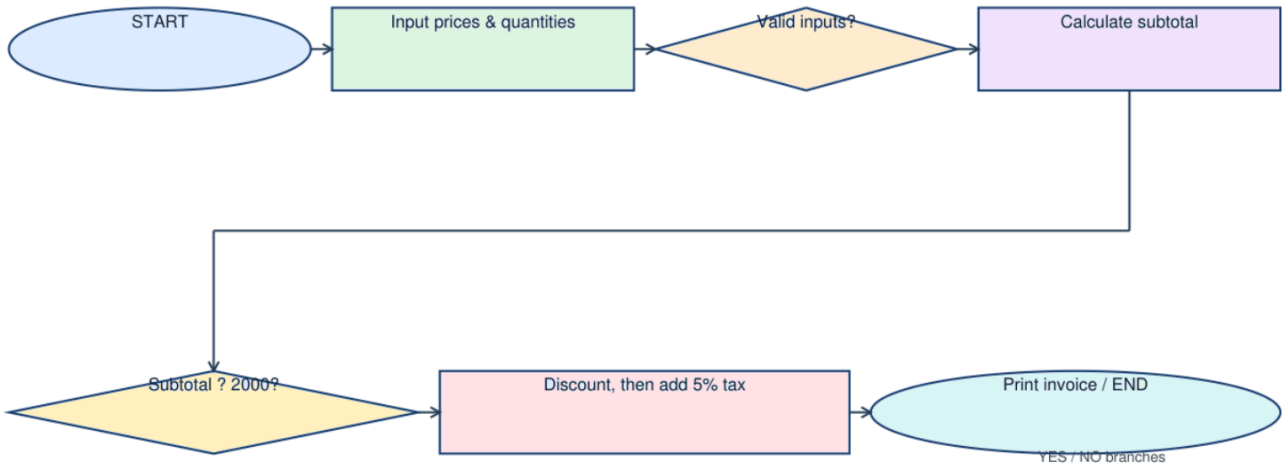
• Calculate each line total and subtotal.

• Apply the threshold discount.

• Calculate tax on the discounted subtotal.

• Display the itemized bill and final amount.

5. Flowchart



6. Pseudocode

```
BEGIN
  subtotal = 0
  REPEAT for 3 products
      INPUT price, quantity
      IF price <= 0 OR quantity <= 0 THEN mark input invalid
      ELSE line_total = price * quantity; subtotal = subtotal + line_total
  END REPEAT
  IF input invalid THEN PRINT error
  ELSE
      IF subtotal >= 2000 THEN discount = subtotal * 0.10 ELSE discount = 0
      taxable = subtotal - discount
      tax = taxable * 0.05
```

```
      final = taxable + tax
      PRINT item totals, subtotal, discount, tax, final
  END IF
END
```

7. Python Source Code

```
subtotal = 0
valid = True
items = []

for i in range(1, 4):
    print("Product", i)
    price = float(input("  Enter unit price: "))
    quantity = int(input("  Enter quantity: "))

    if price <= 0 or quantity <= 0:
        valid = False
    else:
        line_total = price * quantity
        items.append((price, quantity, line_total))
        subtotal += line_total

if not valid:
    print("Invalid price or quantity. Please enter positive values.")
else:
    if subtotal >= 2000:
        discount = subtotal * 0.10
    else:
        discount = 0

    taxable_amount = subtotal - discount
    tax = taxable_amount * 0.05
    final_amount = taxable_amount + tax

    print("\n----- RETAIL INVOICE -----")
    for i, item in enumerate(items, start=1):
        print("Product", i, "line total: Rs.", round(item[2], 2))
    print("Subtotal: Rs.", round(subtotal, 2))
    print("Discount: Rs.", round(discount, 2))
    print("Tax (5%): Rs.", round(tax, 2))
    print("Final amount: Rs.", round(final_amount, 2))
```

8. Test Cases

Three price × quantity pairs	Expected result
500×1, 300×1, 200×1	Subtotal ■1,000; discount ■0; tax ■50; final ■1,050
1000×1, 700×1, 500×1	Subtotal ■2,200; discount ■220; tax ■99; final ■2,079
100×0, 200×1, 300×1	Invalid price or quantity

9. Sample Output

```
Product 1
  Enter unit price: 1000
  Enter quantity: 1
Product 2
  Enter unit price: 700
  Enter quantity: 1
Product 3
  Enter unit price: 500
  Enter quantity: 1

----- RETAIL INVOICE -----
Product 1 line total: Rs. 1000.0
Product 2 line total: Rs. 700.0
Product 3 line total: Rs. 500.0
Subtotal: Rs. 2200.0
Discount: Rs. 220.0
Tax (5%): Rs. 99.0
Final amount: Rs. 2079.0
```

10. Conclusion

The program uses a loop, list storage, arithmetic, and conditions to create an itemized bill with a discount and tax.

Case Study 5: Examination Result and Grace-Mark Validation

1. Problem Statement

Read marks for five subjects, check the pass requirement, apply a limited grace-mark rule when eligible, and print the final result.

2. Objectives

• Store marks in a list.

• Check subject-wise pass criteria.

• Apply grace marks without exceeding maximum marks.

• Display total, average, and result.

3. Assumptions

• There are five subjects, each out of 100.

• The pass mark is 35 in each subject.

• If exactly one subject has marks from 33 to 34, add only enough grace marks to reach 35.

• Grace marks cannot increase any subject above 100.

• Marks must be between 0 and 100.

4. Algorithm

• Read five subject marks into a list.

• Validate every mark.

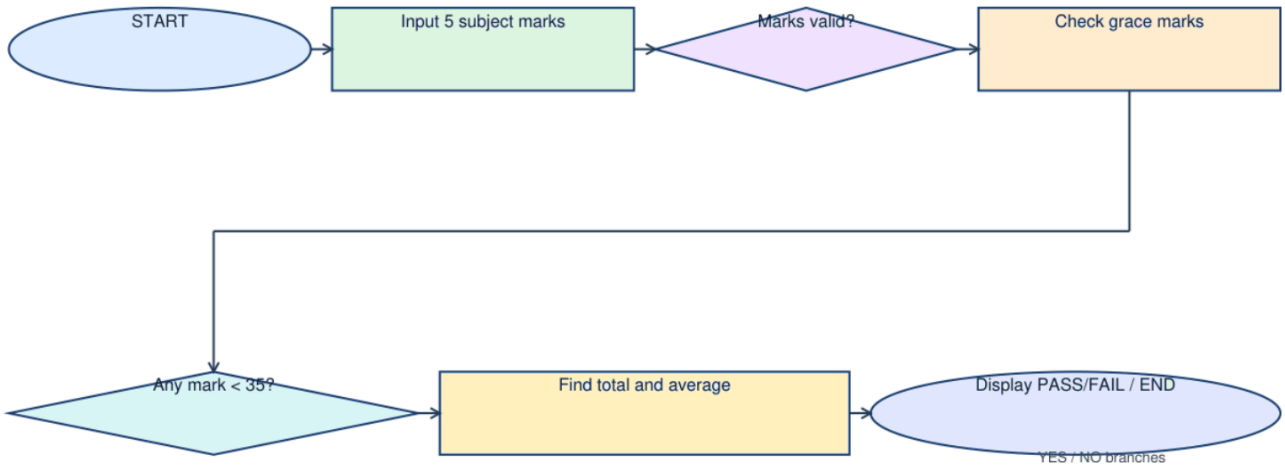
• Find subjects below 35.

• If exactly one subject has 33 or 34 marks, raise it to 35 and record grace marks.

• If any subject is still below 35, result is Fail; otherwise Pass.

• Calculate total and average and display the result.

5. Flowchart



6. Pseudocode

```
BEGIN
  INPUT 5 marks into list
  IF any mark < 0 OR mark > 100 THEN PRINT "Invalid marks"
  ELSE
      find subjects with marks < 35
      IF exactly one subject has 33 or 34 THEN set it to 35
```

```
      IF any mark < 35 THEN result = "Fail" ELSE result = "Pass"
      total = sum(marks); average = total / 5
      PRINT marks, total, average, result
  END IF
END
```

7. Python Source Code

```
marks = []
for i in range(1, 6):
    mark = int(input("Enter marks for subject " + str(i) + ": "))
    marks.append(mark)

if any(mark < 0 or mark > 100 for mark in marks):
    print("Invalid marks. Each mark must be from 0 to 100.")
else:
    grace_used = 0
    near_pass = [i for i, mark in enumerate(marks) if 33 <= mark <= 34]

    if len(near_pass) == 1:
        index = near_pass[0]
        grace_used = 35 - marks[index]
        marks[index] = 35

    total = sum(marks)
    average = total / len(marks)

    if all(mark >= 35 for mark in marks):
        result = "PASS"
    else:
        result = "FAIL"

    print("Final marks:", marks)
    print("Grace marks used:", grace_used)
    print("Total:", total, "/ 500")
    print("Average:", round(average, 2))
    print("Result:", result)
```

8. Test Cases

Marks	Expected result
70, 60, 33, 80, 75	One grace mark; final marks include 35; Pass
70, 60, 20, 80, 75	No grace; Fail
101, 50, 50, 50, 50	Invalid marks

9. Sample Output

```
Enter marks for subject 1: 70
Enter marks for subject 2: 60
Enter marks for subject 3: 33
Enter marks for subject 4: 80
Enter marks for subject 5: 75
Final marks: [70, 60, 35, 80, 75]
Grace marks used: 2
Total: 320 / 500
Average: 64.0
Result: PASS
```

10. Conclusion

The program demonstrates lists, loops, validation, and a clearly defined grace-mark rule. The rule is an assignment assumption, not an official university policy.

Case Study 6: Attendance Recovery Computation

1. Problem Statement

Calculate current attendance percentage and determine the minimum number of consecutive classes a student must attend to reach the required percentage.

2. Objectives

• Calculate attendance percentage.

• Find the minimum future classes needed to reach the threshold.

• Show whether the student is already eligible.

3. Assumptions

• Attendance eligibility threshold is 75%.

• Classes attended cannot exceed total classes.

• The student attends every upcoming class.

• If total classes are zero, percentage is treated as 0%.

4. Algorithm

• Read classes held and classes attended.

• Validate the inputs.

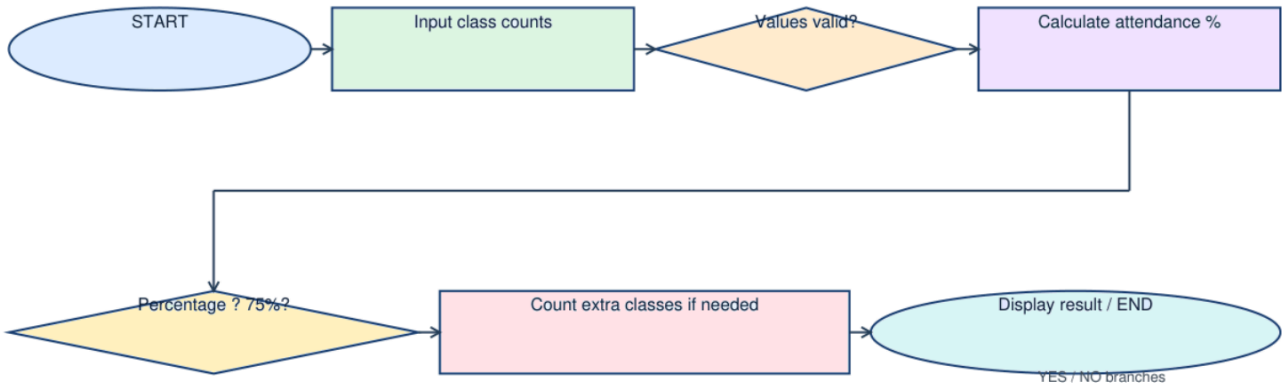
• Calculate current attendance percentage.

• If percentage is at least 75%, report eligible.

• Otherwise, repeatedly add one attended class and one total class until attendance reaches 75%.

• Display the minimum number of additional classes.

5. Flowchart



6. Pseudocode

```
BEGIN
  INPUT total, attended
  IF total < 0 OR attended < 0 OR attended > total THEN PRINT error
  ELSE
      IF total = 0 THEN percentage = 0 ELSE percentage = attended / total * 100
      extra = 0
      WHILE percentage < 75
          attended = attended + 1
          total = total + 1
          extra = extra + 1
          percentage = attended / total * 100
      END WHILE
      PRINT percentage, extra
  END IF
END
```

7. Python Source Code

```
total = int(input("Total classes held: "))
attended = int(input("Classes attended: "))

if total < 0 or attended < 0 or attended > total:
    print("Invalid attendance data")
else:
    if total == 0:
        percentage = 0
    else:
        percentage = attended / total * 100

    current_percentage = percentage
    extra_classes = 0

    while percentage < 75:
        attended += 1
        total += 1
        extra_classes += 1
        percentage = attended / total * 100

    print("Current attendance:", round(current_percentage, 2), "%")
    if extra_classes == 0:
        print("You are already eligible.")
    else:
        print("Additional consecutive classes required:", extra_classes)
        print("Attendance after those classes:", round(percentage, 2), "%")
```

8. Test Cases

Total, attended	Expected result
20, 15	75%; already eligible; 0 extra classes
20, 10	50%; 20 additional classes required
10, 12	Invalid attendance data

9. Sample Output

```
Total classes held: 20
Classes attended: 10
Current attendance: 50.0 %
Additional consecutive classes required: 20
Attendance after those classes: 75.0 %
```

10. Conclusion

The program uses a while loop to calculate the minimum number of future classes needed to meet the attendance threshold.

Case Study 7: ATM Transaction Sequence Simulation

1. Problem Statement

Simulate a sequence of ATM deposits and withdrawals, reject invalid transactions, and display the final balance and transaction summary.

2. Objectives

• Process deposits and withdrawals.

• Prevent withdrawals that exceed the balance or daily limit.

• Count successful and rejected transactions.

3. Assumptions

• Opening balance is ■5,000.

• The withdrawal limit for this simulation is ■3,000 per transaction.

• Transactions are entered as D for deposit or W for withdrawal; X ends input.

• Transaction amounts must be positive.

4. Algorithm

• Set balance to ■5,000 and counters to zero.

• Read a transaction type until X is entered.

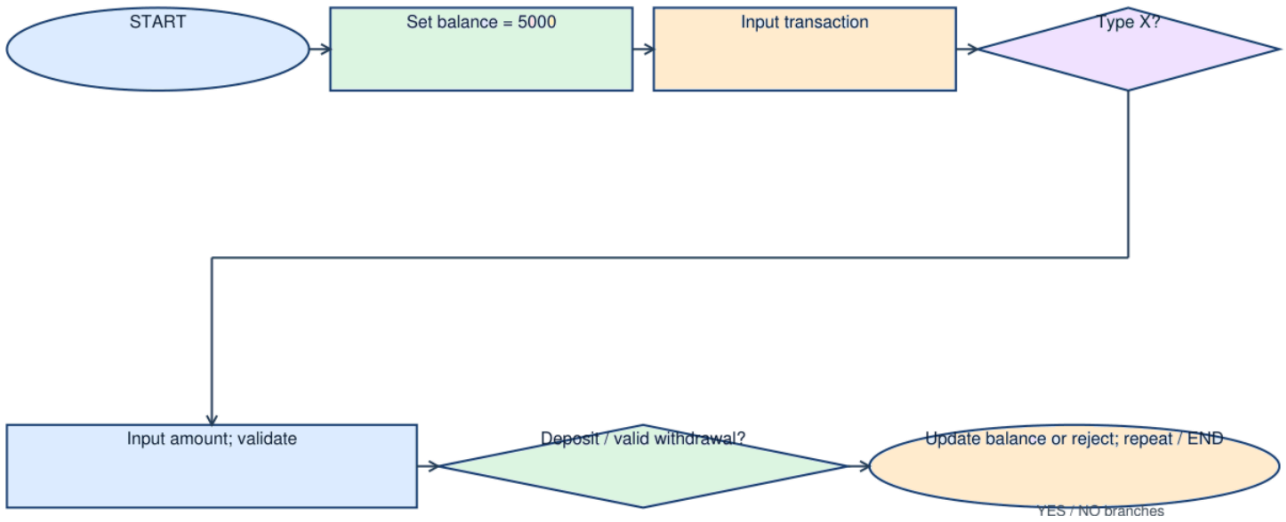
• Read the amount and validate it.

• For deposits, add the amount to balance.

• For withdrawals, check balance and the transaction limit before deducting.

• Update successful/rejected counts and display the final summary.

5. Flowchart



6. Pseudocode

```
BEGIN
  balance = 5000; successful = 0; rejected = 0
  REPEAT
      INPUT type
      IF type = "X" THEN EXIT LOOP
      INPUT amount
      IF amount <= 0 THEN rejected = rejected + 1
      ELSE IF type = "D" THEN balance = balance + amount; successful++
      ELSE IF type = "W" AND amount <= balance AND amount <= 3000
          balance = balance - amount; successful++
```

```
      ELSE rejected = rejected + 1
  END REPEAT
  PRINT balance, successful, rejected
END
```

7. Python Source Code

```
balance = 5000
successful = 0
rejected = 0

while True:
    transaction = input("Enter D for deposit, W for withdrawal, X to exit: ").upper()

    if transaction == "X":
        break

    if transaction not in ("D", "W"):
        print("Invalid transaction type")
        rejected += 1
        continue

    amount = float(input("Enter amount: "))

    if amount <= 0:
        print("Rejected: amount must be positive")
        rejected += 1
    elif transaction == "D":
        balance += amount
        successful += 1
        print("Deposit successful")
    elif amount > balance:
        print("Rejected: insufficient balance")
        rejected += 1
    elif amount > 3000:
        print("Rejected: withdrawal limit is Rs. 3000")
        rejected += 1
    else:
        balance -= amount
        successful += 1
        print("Withdrawal successful")

print("\n----- ATM SUMMARY -----")
print("Final balance: Rs.", round(balance, 2))
print("Successful transactions:", successful)
print("Rejected transactions:", rejected)
```

8. Test Cases

Transaction sequence	Expected result
D 1000; W 2000; X	Final balance ■4,000; 2 successful; 0 rejected
W 6000; X	Final balance ■5,000; 0 successful; 1 rejected
W 3500; X	Final balance ■5,000; 0 successful; 1 rejected

9. Sample Output

```
Enter D for deposit, W for withdrawal, X to exit: D
Enter amount: 1000
Deposit successful
Enter D for deposit, W for withdrawal, X to exit: W
Enter amount: 2000
Withdrawal successful
Enter D for deposit, W for withdrawal, X to exit: X

----- ATM SUMMARY -----
Final balance: Rs. 4000.0
Successful transactions: 2
Rejected transactions: 0
```

10. Conclusion

The program simulates basic ATM transactions using a while loop, conditional statements, and counters. It is an educational simulation, not a real banking system.

Case Study 8: Warehouse Inventory Reconciliation

1. Problem Statement

Compare opening stock, received stock, dispatched stock, and recorded closing stock for several products, and identify discrepancies and replenishment needs.

2. Objectives

• Calculate expected closing stock.

• Compare expected and recorded stock.

• Identify shortages and replenishment quantities.

3. Assumptions

• Three products are processed.

• Expected closing stock = opening stock + received stock – dispatched stock.

• All quantities must be non-negative.

• A target stock of 20 units is used to calculate replenishment need.

4. Algorithm

• Read opening, received, dispatched, and recorded closing quantities for each product.

• Validate that all quantities are non-negative.

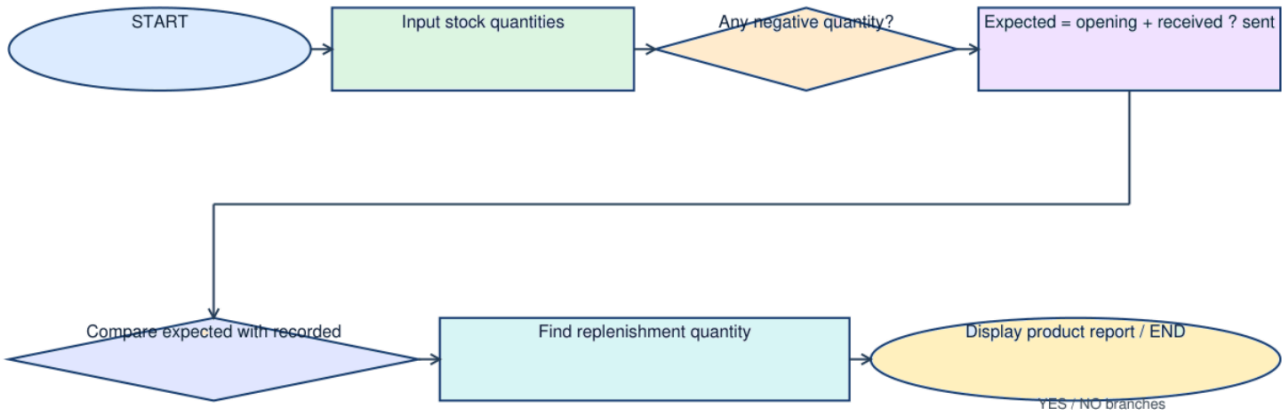
• Calculate expected closing stock.

• Compare expected stock with recorded closing stock.

• Calculate replenishment needed to reach 20 units.

• Display the reconciliation for each product.

5. Flowchart



6. Pseudocode

```
BEGIN
  FOR each product
      INPUT opening, received, dispatched, recorded_closing
      IF any quantity < 0 THEN PRINT invalid record
      ELSE
          expected = opening + received - dispatched
          difference = recorded_closing - expected
          replenishment = MAX(0, 20 - expected)
          PRINT expected, difference, replenishment
      END IF
  END FOR
END
```

7. Python Source Code

```
products = ["Notebook", "Pen", "File"]

for product in products:
    print("\nProduct:", product)
    opening = int(input("Opening stock: "))
    received = int(input("Quantity received: "))
    dispatched = int(input("Quantity dispatched: "))
    recorded_closing = int(input("Recorded closing stock: "))

    if opening < 0 or received < 0 or dispatched < 0 or recorded_closing < 0:
        print("Invalid stock record")
        continue

    expected_closing = opening + received - dispatched
    difference = recorded_closing - expected_closing
    replenishment = max(0, 20 - expected_closing)

    print("Expected closing stock:", expected_closing)
    if difference == 0:
        print("Reconciliation: Match")
    else:
        print("Discrepancy (recorded - expected):", difference)

    print("Replenishment needed to reach 20 units:", replenishment)
```

8. Test Cases

Opening, received, dispatched, recorded closing	Expected result
10, 15, 5, 20	Expected 20; Match; replenishment 0
8, 5, 3, 12	Expected 10; discrepancy +2; replenishment 10
5, 0, 8, 0	Expected -3; report indicates invalid business situation; review dispatched quantity

9. Sample Output

```
Product: Notebook
Opening stock: 10
Quantity received: 15
Quantity dispatched: 5
Recorded closing stock: 20
Expected closing stock: 20
Reconciliation: Match
Replenishment needed to reach 20 units: 0
```

10. Conclusion

The program uses arithmetic, loops, lists, and comparisons to reconcile stock records. In a real system, dispatches should also be checked against available stock.

Case Study 9: Advanced Numeric Pattern Generator

1. Problem Statement

Generate a hollow diamond pattern using nested loops and validate the requested size.

2. Objectives

• Use nested loops to print a symmetric pattern.

• Create a hollow diamond using spaces and star characters.

• Reject dimensions outside the allowed range.

3. Assumptions

• The input size is an integer from 1 to 9.

• The pattern uses asterisks (*) for the boundary.

• The upper half includes the middle row; the lower half mirrors it without repeating the middle row.

4. Algorithm

• Read the size n.

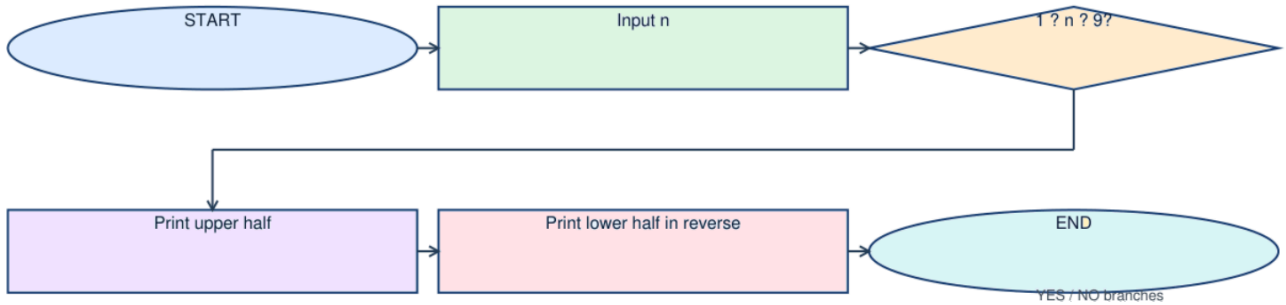
• If n is outside 1–9, display an error.

• For each row in the upper half, print leading spaces, a first star, inner spaces if needed, and a last star.

• Repeat rows in reverse order for the lower half, excluding the middle row.

• Stop.

5. Flowchart



6. Pseudocode

```
BEGIN
  INPUT n
  IF n < 1 OR n > 9 THEN PRINT "Invalid size"
  ELSE
      FOR row = 1 TO n
          PRINT n-row leading spaces
          PRINT first star
          IF row > 1 THEN PRINT 2*row-3 inner spaces and last star
      END FOR
      FOR row = n-1 DOWN TO 1
          PRINT n-row leading spaces
          PRINT first star
          IF row > 1 THEN PRINT 2*row-3 inner spaces and last star
      END FOR
  END IF
END
```

7. Python Source Code

```
n = int(input("Enter diamond size (1 to 9): "))

if n < 1 or n > 9:
    print("Invalid size. Enter a number from 1 to 9.")
else:
```

```
    # Upper half, including the middle row
    for row in range(1, n + 1):
        print(" " * (n - row), end="")
        if row == 1:
            print("*")
        else:
            print("*" + " " * (2 * row - 3) + "*")

    # Lower half
    for row in range(n - 1, 0, -1):
        print(" " * (n - row), end="")
        if row == 1:
            print("*")
        else:
            print("*" + " " * (2 * row - 3) + "*")
```

8. Test Cases

Size	Expected result
1	One star
3	Hollow diamond with 5 rows
0	Invalid size

9. Sample Output

```
Enter diamond size (1 to 9): 3
  *
 * *
*   *
 * *
  *
```

10. Conclusion

The program uses nested loops, string repetition, and conditions to generate a hollow diamond pattern.

Case Study 10: Student Marks Matrix Analysis

1. Problem Statement

Store marks for multiple students and subjects in a nested list, calculate each student's total and average, calculate subject averages, and identify the top performer.

2. Objectives

• Represent marks using a nested list.

• Calculate row-wise and column-wise statistics.

• Identify the student with the highest total.

• Identify subjects with average marks below a threshold.

3. Assumptions

• Three students and three subjects are used for demonstration.

• Each mark is between 0 and 100.

• Subject averages below 50 are flagged as low.

• The top performer is the student with the highest total; ties are resolved by keeping the first student listed.

4. Algorithm

• Create lists of student names and subject names.

• Read marks into a nested list.

• Validate marks.

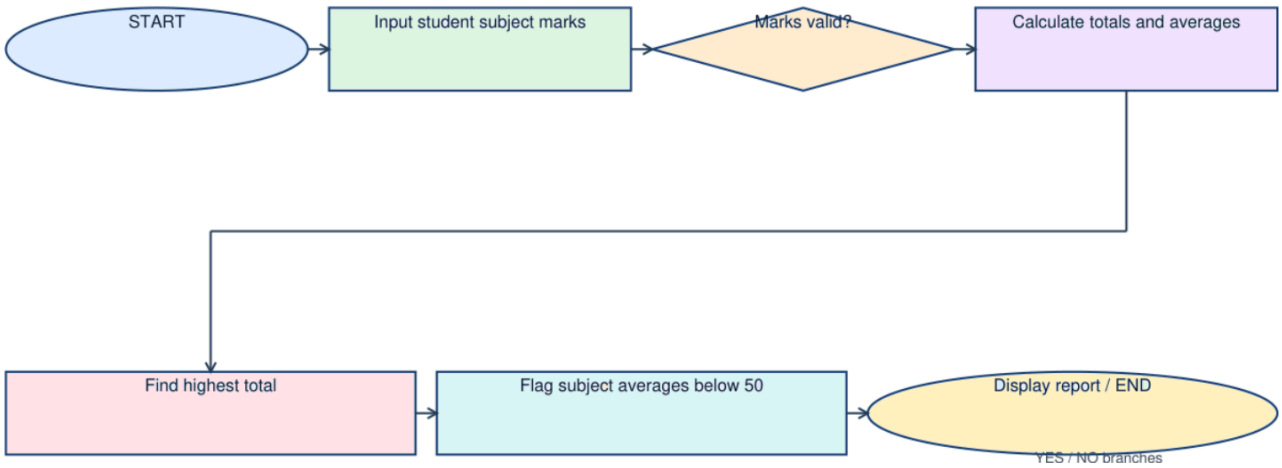
• Calculate each student's total and average.

• Calculate the average for each subject.

• Find the highest total and flag subjects with average below 50.

• Display the report.

5. Flowchart



6. Pseudocode

```
BEGIN
  INPUT marks matrix
  IF any mark < 0 OR mark > 100 THEN PRINT error
  ELSE
      FOR each student row
          total = SUM(row); average = total / number_of_subjects
      END FOR
      FOR each subject column
```

```
            subject_average = SUM(column) / number_of_students
        END FOR
        top_student = student with maximum total
        PRINT student statistics, subject averages, top student
    END IF
END
```

7. Python Source Code

```
students = ["Asha", "Bharathi", "Charan"]
subjects = ["Python", "Maths", "English"]

marks = [
    [85, 78, 90],
    [72, 88, 68],
    [45, 55, 60]
]

if any(mark < 0 or mark > 100 for row in marks for mark in row):
    print("Invalid marks")
else:
    totals = []
    print("STUDENT RESULTS")
    for i in range(len(students)):
        total = sum(marks[i])
        average = total / len(subjects)
        totals.append(total)
        print(students[i], "- Marks:", marks[i],
              "Total:", total, "Average:", round(average, 2))

    print("\nSUBJECT AVERAGES")
    for j in range(len(subjects)):
        subject_total = 0
        for i in range(len(students)):
            subject_total += marks[i][j]
        subject_average = subject_total / len(students)
        print(subjects[j] + ":", round(subject_average, 2))
        if subject_average < 50:
            print("  Warning: low average in", subjects[j])

    top_index = totals.index(max(totals))
    print("\nTop performer:", students[top_index])
    print("Highest total:", totals[top_index], "/", len(subjects) * 100)
```

8. Test Cases

Marks matrix	Expected result
[[85,78,90],[72,88,68],[45,55,60]]	Totals: Asha 253; Bharathi 228; Charan 160; top performer Asha
Any mark is -1 or 101	Invalid marks

9. Sample Output

```
STUDENT RESULTS
Asha - Marks: [85, 78, 90] Total: 253 Average: 84.33
Bharathi - Marks: [72, 88, 68] Total: 228 Average: 76.0
Charan - Marks: [45, 55, 60] Total: 160 Average: 53.33

SUBJECT AVERAGES
Python: 67.33
Maths: 73.67
English: 72.67

Top performer: Asha
Highest total: 253 / 300
```

10. Conclusion

The program demonstrates nested lists, nested loops, row-wise and column-wise calculations, and selection of a top performer.

Final Submission Checklist

- ■ Check whether your faculty expects handwritten or drawn flowchart symbols; redraw the text flowcharts if needed.
- ■ Run every program and capture your own output screenshots as required by the assignment.
- ■ Verify all sample assumptions with your faculty and change rates or thresholds if instructed.
- ■ Save the PDF and Python files in your GitHub repository named Python-Activity-Based-Learning.
- ■ Include your repository link when submitting the assignment.